

Network Traffic Analysis & Threat Detection Lab

Packet-level analysis using Kali Linux, Wireshark and Metasploitable 2

Student	Pritam Bose
Program	B.Tech Computer Science Engineering
Environment	Kali Linux + Metasploitable 2 (isolated lab)
Focus	Network traffic observation, reconnaissance detection and protocol analysis

Prepared by Pritam Bose

1. Introduction

This project focuses on practical network traffic analysis and basic threat detection in a controlled cybersecurity laboratory. The aim was to generate and observe different traffic types between a Kali Linux system and a deliberately vulnerable Metasploitable 2 virtual machine, then use Wireshark to identify communication patterns at packet level.

The project compares normal application communication, reconnaissance activity, DNS resolution and encrypted remote-access traffic.

2. Objective

- Capture and inspect network packets using Wireshark.
- Understand the TCP three-way handshake and HTTP request/response flow.
- Identify the packet pattern produced by a TCP SYN port scan.
- Observe DNS query and response traffic.
- Observe SSH traffic and identify encrypted application packets.
- Compare normal communication patterns with reconnaissance behaviour.
- Document observations as reproducible evidence from an isolated lab.

3. Scope

Testing was limited to the authorized virtual laboratory. The project covers HTTP, TCP SYN scanning, DNS and SSH traffic. No production or third-party systems were used.

4. Lab Environment

Component	Details
Analysis host	Kali Linux
Target	Metasploitable 2
Kali IP	192.168.202.131
Target IP	192.168.202.130
Interface	eth0
Packet analysis	Wireshark
Reconnaissance	Nmap

5. Tools Used

- Kali Linux — security testing and analysis environment.
- Wireshark — packet capture and protocol analysis.
- Nmap — network discovery and TCP SYN scanning.
- Metasploitable 2 — intentionally vulnerable virtual lab target.

- nslookup — DNS query generation.
- SSH client — generation and observation of SSHv2 traffic.

6. Methodology

Start the isolated Kali Linux and Metasploitable 2 virtual machines.

Identify the target IP and verify connectivity.

Capture traffic on eth0 with Wireshark.

Generate one controlled traffic type at a time.

Apply a suitable Wireshark display filter.

Inspect packet fields and communication direction.

Save screenshots as evidence.

Interpret and compare the observations.

7. Reconnaissance

The Metasploitable host was identified as 192.168.202.130. Nmap reconnaissance showed multiple exposed TCP services, including HTTP on port 80 and SSH on port 22.

Command: `nmap -sS 192.168.202.130`

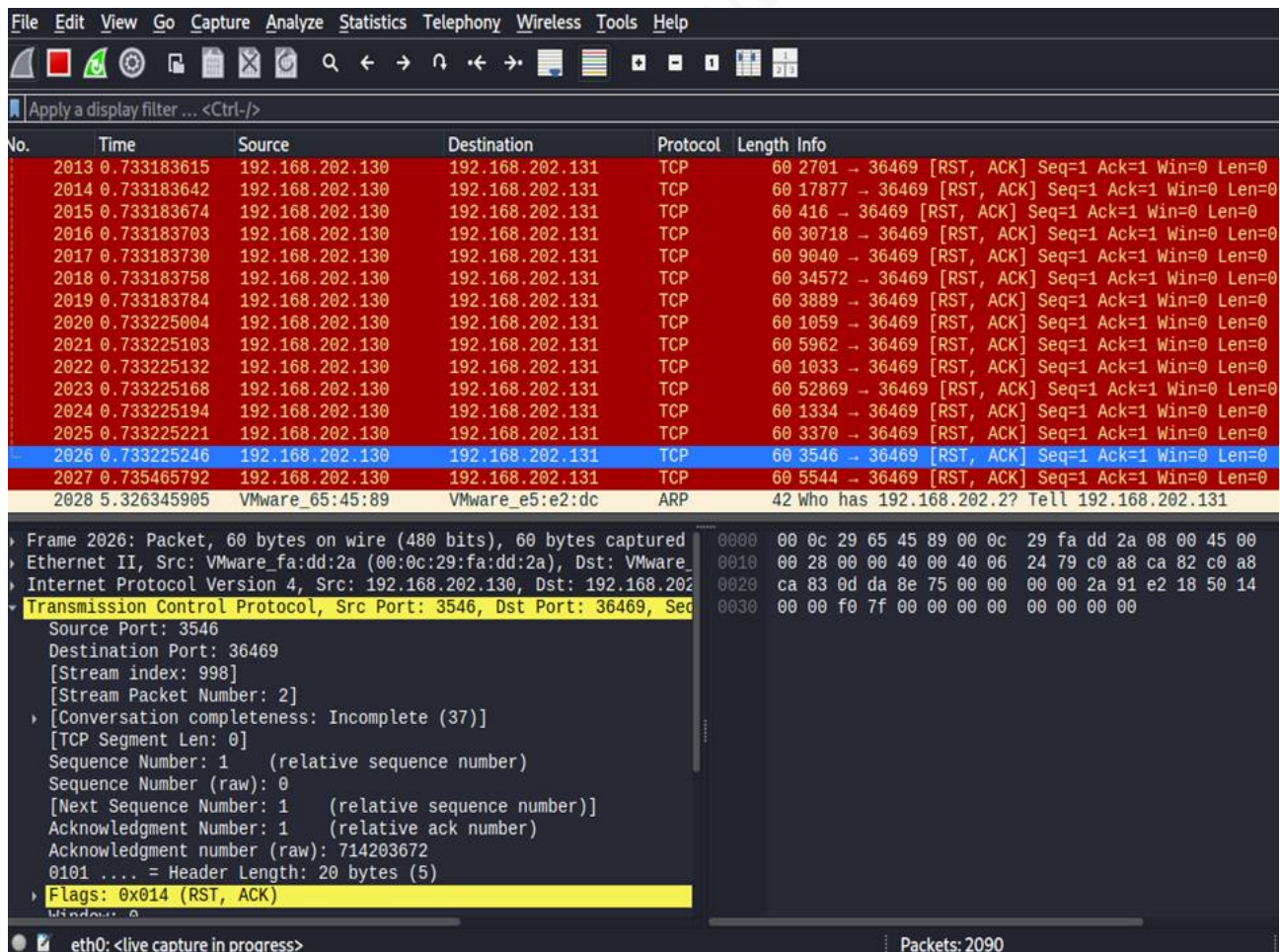


Figure 1: Nmap reconnaissance result for the Metasploitable lab host.

8. Experiment 1 — Normal HTTP Traffic Analysis

A baseline HTTP exchange was captured between Kali Linux and the Metasploitable web server.

8.1 Procedure

- A. Start capture on eth0.
 - B. Generate an HTTP request to 192.168.202.130.
 - C. Apply: ip.addr == 192.168.202.130 && tcp.port == 80
 - D. Inspect the HTTP GET request.
 - E. Inspect the corresponding HTTP 200 OK response.
- Source: 192.168.202.131
 - Destination: 192.168.202.130
 - Destination port: 80
 - Method: GET
 - URI: /
 - Version: HTTP/1.1

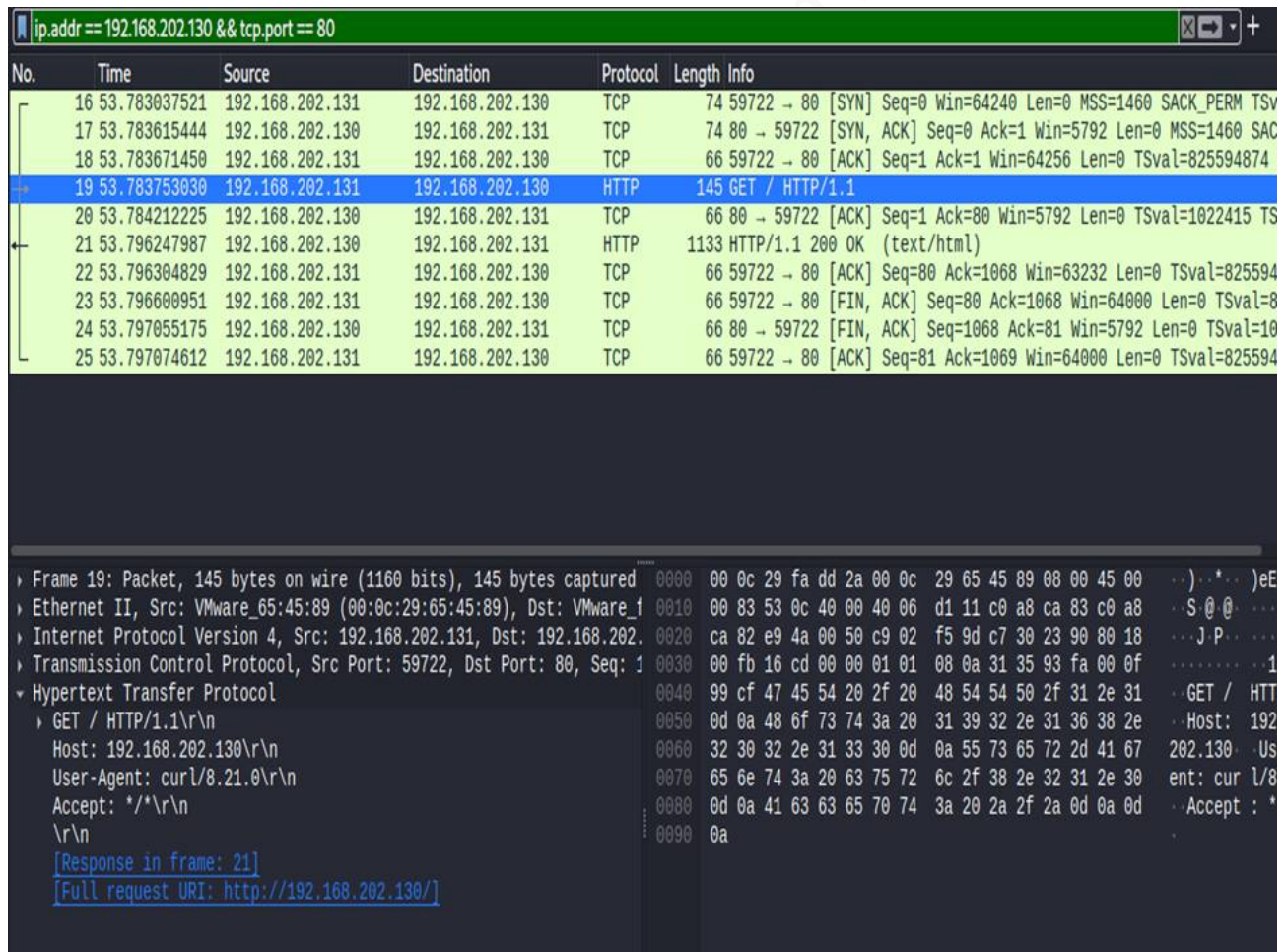


Figure 2: HTTP GET request captured in Wireshark.

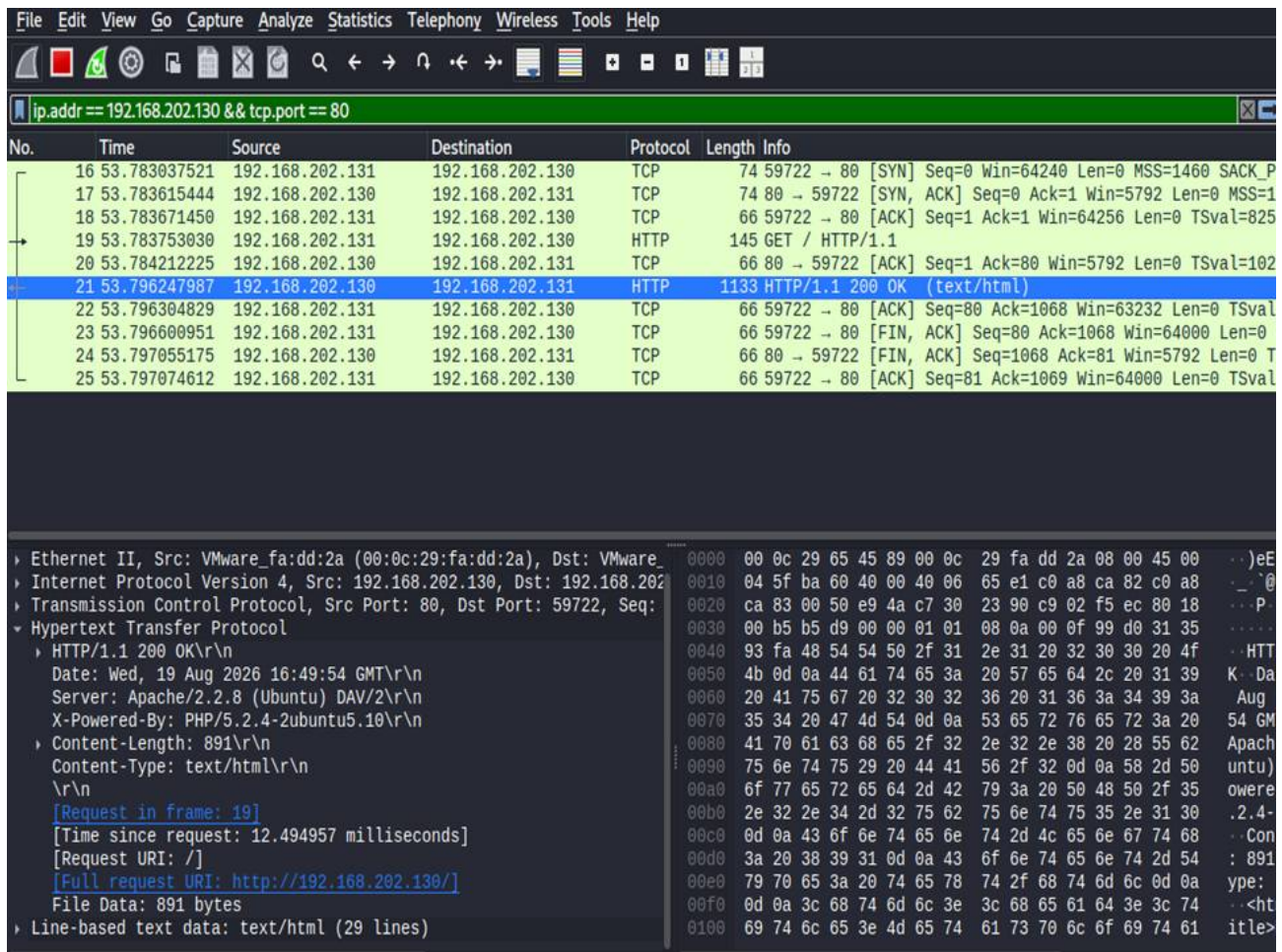


Figure 3: HTTP 200 OK response captured in Wireshark.

8.2 Observation

The packet sequence showed the TCP three-way handshake followed by an HTTP GET request and a successful HTTP 200 OK response. This establishes a baseline example of normal web traffic.

9. Experiment 2 — TCP SYN Port Scan Detection

A controlled TCP SYN scan was generated against the authorized Metasploitable target and observed in Wireshark.

9.1 Procedure

- Start a fresh Wireshark capture on eth0.
- Run `nmap -sS 192.168.202.130`.
- Apply: `ip.src == 192.168.202.131 && ip.dst == 192.168.202.130 && tcp.flags.syn == 1 && tcp.flags.ack == 0`

Observe the destination ports associated with SYN probes.

Compare the packet pattern with the Nmap result.

The capture showed repeated SYN packets from 192.168.202.131 to multiple ports on 192.168.202.130, including ports such as 199, 8080, 21, 443, 993, 22, 110, 139 and 5900.

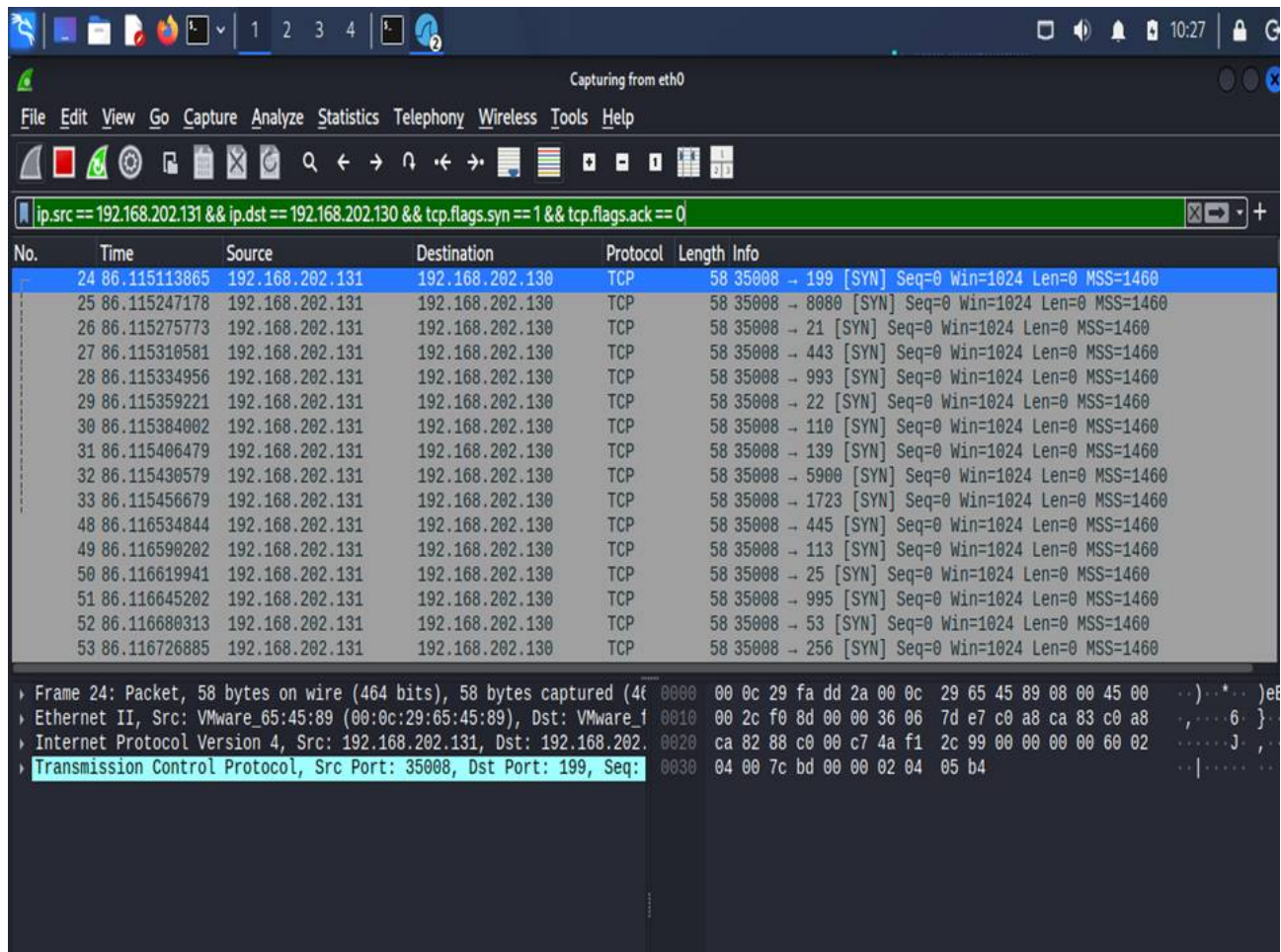


Figure 4: Multiple TCP SYN probes from Kali Linux to different Metasploitable ports.

9.2 Observation

A single source host repeatedly sending TCP SYN packets to many destination ports is consistent with TCP SYN-based port scanning. This shows how reconnaissance can be identified from packet behaviour, not only from terminal output.

10. Experiment 3 — DNS Traffic Analysis

A DNS lookup for example.com was generated from Kali Linux and captured using Wireshark.

10.1 Procedure

- Start a fresh capture on eth0.
- Run `nslookup example.com`.
- Apply the filter: `dns`
- Inspect the DNS query and response.

E. Inspect Transaction ID, flags, query name and record type.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.202.131	192.168.202.2	DNS	71	Standard query 0xd402 A example.com
4	0.036006722	192.168.202.2	192.168.202.131	DNS	103	Standard query response 0xd402 A example.com A 172.66.147.1
5	0.036715327	192.168.202.131	192.168.202.2	DNS	71	Standard query 0xf09 AAAA example.com
6	0.064052677	192.168.202.2	192.168.202.131	DNS	99	Standard query response 0xf09 AAAA example.com AAAA 2606:2800:4000:0000:0000:0000:0000:0000
17	51.202160874	192.168.202.131	192.168.202.2	DNS	71	Standard query 0xd90f A example.com
18	51.207635966	192.168.202.2	192.168.202.131	DNS	103	Standard query response 0xd90f A example.com A 104.20.235.1
19	51.208216536	192.168.202.131	192.168.202.2	DNS	71	Standard query 0x01bf AAAA example.com
20	51.212342996	192.168.202.2	192.168.202.131	DNS	99	Standard query response 0x01bf AAAA example.com AAAA 2606:2800:4000:0000:0000:0000:0000:0000

Frame 1: Packet, 71 bytes on wire (568 bits), 71 bytes captured (568 bits) on interface 0
 Ethernet II, Src: VMware_65:45:89 (00:0c:29:65:45:89), Dst: VMware_08:00:27:00:00:00 (00:0c:29:00:00:00)
 Internet Protocol Version 4, Src: 192.168.202.131, Dst: 192.168.202.2
 User Datagram Protocol, Src Port: 40088, Dst Port: 53
 Domain Name System (query)
 Transaction ID: 0xd402
 Flags: 0x0100 Standard query
 Questions: 1
 Answer RRs: 0
 Authority RRs: 0
 Additional RRs: 0
 Queries
 [Response In: 4]

Figure 5: DNS query for example.com captured in Wireshark.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.202.131	192.168.202.2	DNS	71	Standard query 0xd402 A example.com
4	0.036006722	192.168.202.2	192.168.202.131	DNS	103	Standard query response 0xd402 A example.com A 172.66.147.1
5	0.036715327	192.168.202.131	192.168.202.2	DNS	71	Standard query 0xf09 AAAA example.com
6	0.064052677	192.168.202.2	192.168.202.131	DNS	99	Standard query response 0xf09 AAAA example.com AAAA 2606:2800:4000:0000:0000:0000:0000:0000
17	51.202160874	192.168.202.131	192.168.202.2	DNS	71	Standard query 0xd90f A example.com
18	51.207635966	192.168.202.2	192.168.202.131	DNS	103	Standard query response 0xd90f A example.com A 104.20.235.1
19	51.208216536	192.168.202.131	192.168.202.2	DNS	71	Standard query 0x01bf AAAA example.com
20	51.212342996	192.168.202.2	192.168.202.131	DNS	99	Standard query response 0x01bf AAAA example.com AAAA 2606:2800:4000:0000:0000:0000:0000:0000

Frame 4: Packet, 103 bytes on wire (824 bits), 103 bytes captured (824 bits) on interface 0
 Ethernet II, Src: VMware_e5:e2:dc (00:50:56:e5:e2:dc), Dst: VMware_08:00:27:00:00:00 (00:0c:29:00:00:00)
 Internet Protocol Version 4, Src: 192.168.202.2, Dst: 192.168.202.131
 User Datagram Protocol, Src Port: 53, Dst Port: 40088
 Domain Name System (response)
 Transaction ID: 0xd402
 Flags: 0x8180 Standard query response, No error
 Questions: 1
 Answer RRs: 2
 Authority RRs: 0
 Additional RRs: 0
 Queries
 Answers
 [Request In: 1]
 [Time: 36.006722 milliseconds]

Figure 6: DNS response and packet details captured in Wireshark.

10.2 Observation

The capture showed DNS standard queries from the Kali host to the configured DNS server and corresponding responses. A and AAAA queries for example.com were observed. Standard DNS fields such as the transaction identifier and flags were visible in packet details.

11. Experiment 4 — SSH Encrypted Traffic Analysis

SSH traffic was examined to demonstrate the difference between visible connection metadata and encrypted application data.

11.1 Procedure

- Start a fresh capture on eth0.
- Initiate SSH from Kali Linux to 192.168.202.130.
- Use the legacy host-key option required by the old Metasploitable SSH server.
- Apply: `tcp.port == 22`
- Inspect SSHv2 packets and packet details.

The SSH session was successfully established. Wireshark displayed SSHv2 packets between 192.168.202.131 and 192.168.202.130, with the selected packet identified as encrypted.

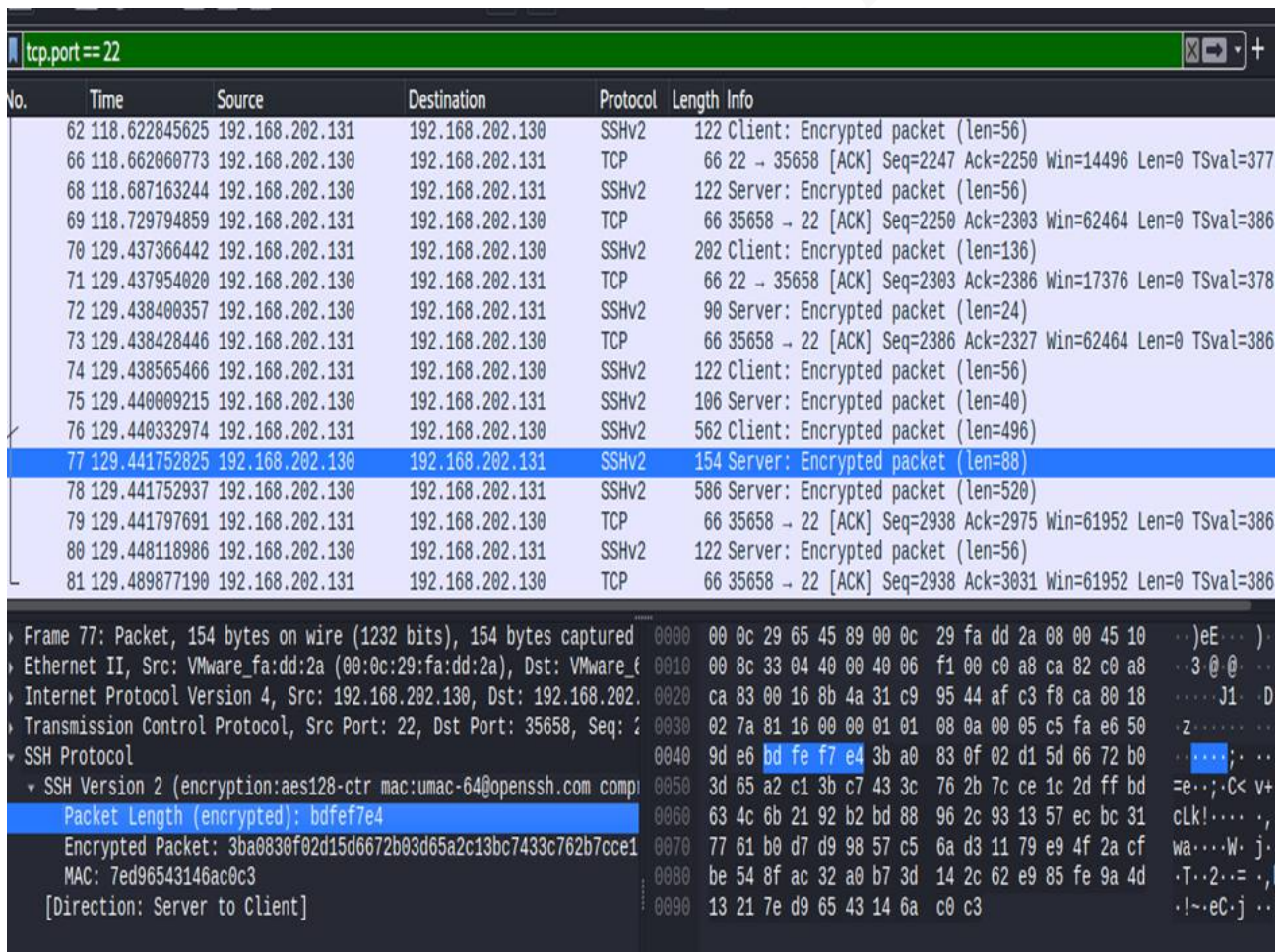


Figure 7: SSHv2 encrypted traffic captured between Kali Linux and Metasploitable.

11.2 Observation

The SSH connection was visible at the transport and protocol level, including the participating hosts and TCP port 22. The application payload was identified as encrypted rather than appearing as readable commands or credentials.

12. Analysis and Comparison

The experiments show that different network activities produce different packet-level characteristics. Normal application protocols generally follow recognizable request-response patterns, while reconnaissance can create repeated connection attempts across many destination ports.

Traffic	Key observation	Interpretation	Security relevance
HTTP	GET followed by HTTP 200 OK	Normal web request/response	Baseline for web traffic
TCP SYN scan	Many SYN probes to different ports	Reconnaissance pattern	Can indicate port scanning
DNS	Query and corresponding response	Name resolution	Useful for DNS monitoring
SSH	SSHv2 encrypted packets on TCP 22	Encrypted remote session	Payload is not directly readable

12.1 Detection Logic

- HTTP: expected TCP establishment followed by an HTTP request and response.
- Port scan: one source generating many SYN attempts against different ports in a short period.
- DNS: correlate queries with responses using transaction IDs and query information.
- SSH: identify TCP 22/SSHv2 traffic while recognizing that application payloads are encrypted.

13. Risk Summary

The lab does not represent a production compromise. The main security observation is that reconnaissance activity can be identified through repeated SYN probes across multiple ports. If the same pattern appeared against a real internal host without authorization, it could warrant investigation.

Activity	Observed pattern	Potential concern	Suggested response
TCP SYN scan	Multiple ports probed from one source	Network reconnaissance	Investigate source and correlate logs
HTTP	Normal GET/200 exchange	Depends on application content	Monitor unusual requests and server logs
DNS	Standard query/response	Abnormal domains or volumes may be suspicious	Monitor DNS patterns
SSH	Encrypted SSHv2 session	Unauthorized remote access if unexpected	Restrict SSH and review auth logs

14. Remediation / Defensive Considerations

- Restrict unnecessary exposed ports and services using firewalls.
- Monitor for repeated SYN probes across many destination ports.

- Use IDS/IPS or network monitoring rules to flag scanning patterns.
- Limit SSH access to trusted hosts or management networks.
- Keep SSH/server software current and securely configured.
- Monitor DNS for unusual query volumes, suspicious domains and unexpected DNS destinations.
- Correlate packet observations with host and authentication logs.

Prepared by Pritam Bose

15. Conclusion

This project provided practical experience in monitoring and analysing network traffic using Wireshark in a controlled lab environment consisting of Kali Linux and Metasploitable 2. The experiments demonstrated how network communication can be observed at the packet level and how different protocols and connection behaviours can be identified from captured traffic. During the experiments, HTTP communication was analysed to observe TCP connection establishment, HTTP requests and server responses. TCP traffic generated during port-scanning activity was examined to identify connection attempts, destination ports and TCP flags such as SYN and RST/ACK. DNS traffic was also analysed to understand how domain-name queries and responses are exchanged between a host and a DNS server. Finally, SSH traffic was captured and analysed to demonstrate how an encrypted communication session appears in a packet capture. The experiments also showed an important limitation of packet analysis: **not every captured packet is automatically malicious or suspicious**. Traffic needs to be interpreted according to its source, destination, protocol, port, flags, sequence of events and overall behaviour. For example, repeated SYN packets directed toward multiple ports can indicate scanning activity, while normal DNS queries and an established SSH session represent legitimate network behaviour in the lab. Overall, the project strengthened practical understanding of **packet capture, protocol identification, TCP behaviour, port activity, DNS communication, encrypted traffic and basic threat-detection concepts**. It also demonstrated how Wireshark can be used as an investigation tool to move from raw network packets to a reasoned security conclusion. The knowledge gained from this lab can be applied to future security investigations involving network monitoring, suspicious connection analysis, intrusion detection and incident response.

16. References

- Wireshark Foundation. *Wireshark User's Guide*. [Wireshark User's Guide](#)
- Wireshark Foundation. *Wireshark Documentation*. [Wireshark Documentation](#)
- Nmap Project. *Nmap Reference Guide*. [Nmap Reference Guide](#)
- Nmap Project. *Nmap Network Scanning*. [Nmap Network Scanning](#)
- Metasploit / Rapid7. *Metasploitable 2*. [Metasploitable 2 Documentation](#)
- RFC Editor. *RFC 793 — Transmission Control Protocol*. [RFC 793](#)
- RFC Editor. *RFC 768 — User Datagram Protocol*. [RFC 768](#)
- RFC Editor. *RFC 1034 — Domain Names: Concepts and Facilities*. [RFC 1034](#).

Prepared by Pritam Bose